

□ Titan Robot

Titan Robot is an open-source, ROS 2-based mobile robot platform designed for students and educators to learn about ROS 2, mobile robotics, and autonomous navigation. Inspired by platforms like TurtleBot, Titan Robot provides a hands-on experience with both hardware and software, making it ideal for research, prototyping, and education.

□ Table of Contents

1. Overview
 2. Hardware Architecture
 3. Electronics & Power
 4. Software Architecture
 5. Robot Bringup
 6. Mapping & Localization
 7. Navigation
 8. Network Setup
 9. Development & Customization
 10. Troubleshooting
 11. References
-

Overview

Titan Robot is a differential-drive mobile robot built around a Raspberry Pi 4B (8GB RAM) running Ubuntu and ROS 2 Humble. It features a SLiDAR C1 for 2D mapping and navigation, an ESP32 microcontroller for low-level motor control (with PID and dead reckoning), and robust power management with an 11.1V 2500mAh NMC battery. The robot is designed for easy assembly, extensibility, and real-world experimentation.

Hardware Architecture

1. Main Controller: Raspberry Pi 4B (8GB RAM)

- **Role:** Runs ROS 2 nodes, handles high-level logic, SLAM, navigation, and user interfaces.
- **OS:** Ubuntu 22.04 LTS (64-bit recommended).
- **Connectivity:** Wi-Fi/Ethernet for remote access and SSH.
- **Interfaces:** USB (for SLiDAR and ESP32), GPIO (optional for expansion).

2. Motor Controller: ESP32 MCU

- **Role:** Handles real-time motor control, PID velocity loops, encoder feedback, and dead reckoning.
- **Firmware:** Custom C++/Arduino code implementing:
 - PID control for smooth and accurate wheel motion.
 - Odometry calculation using wheel encoders.
 - Serial communication protocol with Raspberry Pi.
- **Communication:** Serial (UART/USB) to Raspberry Pi, using a custom ROS 2 node for data exchange.

3. SLiDAR C1

- **Role:** 360° 2D LIDAR for mapping, localization, and obstacle detection.
- **Interface:** USB.
- **Integration:** ROS 2 driver publishes `/scan` topic for SLAM and navigation.

4. Chassis & Drive

- **Motors:** Two DC gear motors with encoders (differential drive).
- **Wheels:** Rubber wheels for traction.
- **Caster:** Passive caster wheel for balance.
- **Frame:** Custom or off-the-shelf chassis, with mounting for Pi, battery, LIDAR, and electronics.

5. Power System

- **Battery:** 11.1V 2500mAh NMC (Lithium Nickel Manganese Cobalt Oxide).
 - **Features:** Native charging support (charge without removing battery).
 - **Power Distribution:** 5V/3A regulator for Raspberry Pi, 12V for motors.
 - **Safety:** Power button for safe startup/shutdown, fuses for protection.
-

Electronics & Power

- **Wiring:** Use quality wires and connectors for all power and signal lines.
 - **Power Button:** Inline with battery positive, latching or momentary with relay.
 - **Charging:** Dedicated charging port with protection circuitry.
 - **Indicators:** Optional LEDs for power, charging, and status.
-

Software Architecture

1. ROS 2 Humble

- **Core Framework:** All robot logic, drivers, and algorithms run as ROS 2 nodes.
- **Packages Used:**
 - `slam_toolbox` for mapping and localization.
 - `nav2` (Navigation2) for path planning and autonomous navigation.
 - Custom packages for ESP32 communication, teleop, and robot bringup.

2. Communication Stack

- **ESP32 Node:** Handles serial communication, publishes odometry, subscribes to velocity commands.
- **LIDAR Node:** Publishes laser scans.
- **Navigation Stack:** Subscribes to odometry and scan, publishes velocity commands.

3. Teleoperation

- **Keyboard Teleop:** Control robot via `teleop_twist_keyboard`.
 - **RViz2:** Visualize robot, map, and navigation goals.
-

Robot Bringup

1. Workspace Setup

```
mkdir -p ~/titan_ws/src
cd ~/titan_ws/src
git clone https://github.com/MRS111-OS/titan_robot.git
cd ~/titan_ws
colcon build
source install/setup.bash
```

2. Launch Robot

```
ros2 launch titan_bringup titan_bringup.launch.py
```

- Starts URDF, LIDAR, ESP32 node, and RViz2.

Mapping & Localization

Mapping with SLAM Toolbox

1. Launch navigation stack:

```
ros2 launch nav2_bringup navigation_launch.py
```
2. Launch SLAM Toolbox:

```
ros2 launch slam_toolbox online_async_launch.py
```
3. Teleoperate robot to explore:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```
4. Save map in RViz2 using SlamToolboxPlugin.

Localization

1. Edit `mapper_params_online_async.yaml`:

```
mode: localization
map_file_name: "/absolute/path/to/your/map.yaml"
map_start_at_dock: true
```
 2. Launch SLAM Toolbox in localization mode:

```
ros2 launch slam_toolbox online_async_launch.py
```
 3. In RViz2, use 2D Goal Pose tool to send navigation goals.
-

Navigation

- **Navigation2 (Nav2):** Provides autonomous navigation, path planning, and obstacle avoidance.
 - **Inputs:** Odometry from ESP32, laser scans from SLiDAR.
 - **Outputs:** Velocity commands to ESP32.
 - **RViz2:** Set goals and monitor robot state.
-

Network Setup

To find your robot's IP address:

1. **Install nmap:**

```
sudo apt update
sudo apt install nmap
```

2. Scan your network:

```
sudo nmap -sn 192.168.127.0/25
```

- Look for the Raspberry Pi in the output.
-

Development & Customization

- **Hardware Expansion:** Add cameras, IMUs, or other sensors via USB or GPIO.
 - **Software:** Create new ROS 2 nodes for custom behaviors.
 - **Simulation:** Use Gazebo for virtual testing (future support).
-

Troubleshooting

- **No LIDAR data:** Check USB connection and driver node.
 - **No movement:** Check ESP32 firmware, serial connection, and power.
 - **Navigation issues:** Verify map quality, localization, and odometry accuracy.
 - **Power issues:** Ensure battery is charged and power button is ON.
-

References

- ROS 2 Documentation
 - SLAM Toolbox
 - Navigation2
 - SLiDAR C1
 - ESP32 Arduino
 - Raspberry Pi
-

For detailed assembly, wiring diagrams, and firmware examples, see the /docs folder (to be added).

Titan Robot is an open learning platform. Contributions and improvements are welcome!

Titan Robot – Concepts and Key Technologies

PID Control (Proportional-Integral-Derivative)

PID control is a fundamental feedback mechanism used in robotics for precise motion control. In Titan Robot, the ESP32 microcontroller implements a PID controller to regulate the speed and position of the motors based on encoder feedback.

- **Proportional (P):** Reacts to the current error (difference between desired and actual speed/position).
- **Integral (I):** Reacts to the accumulation of past errors, helping eliminate steady-state error.
- **Derivative (D):** Predicts future error based on its rate of change, helping dampen oscillations.

How it works in Titan Robot:

The ESP32 receives velocity commands from ROS 2, reads wheel encoder values, computes the error, and adjusts motor PWM signals to minimize the error, resulting in smooth and accurate movement.

Dead Reckoning

Dead reckoning is a method for estimating the robot's position (odometry) by integrating wheel encoder data over time.

- **Principle:** By measuring how much each wheel has turned, the robot can estimate its change in position and orientation.
- **Limitations:** Errors accumulate over time due to wheel slip, uneven surfaces, or encoder inaccuracies. Therefore, dead reckoning is often combined with external sensors (like LIDAR) for correction.

How it works in Titan Robot:

The ESP32 calculates the robot's pose (x , y , θ) using encoder ticks and sends this odometry data to the Raspberry Pi via serial, where it is published as a ROS 2 topic for use in SLAM and navigation.

SLiDAR C1 (2D LIDAR Sensor)

The SLiDAR C1 is a compact, 360° 2D laser scanner used for mapping, localization, and obstacle detection.

- **Key Features:**
 - **Range:** Up to 12 meters.

- **Scan Rate:** Up to 10 Hz (rotations per second).
 - **Angular Resolution:** $\sim 0.5^\circ$.
 - **Interface:** USB (plug-and-play with ROS 2 drivers).
 - **Lightweight:** Suitable for small robots.
- **How it works:**
The LIDAR emits laser pulses and measures the time it takes for the light to reflect off objects and return. By rotating rapidly, it creates a 2D map of distances to obstacles all around the robot.
 - **Integration with ROS 2:**
The SLiDAR C1 is supported by open-source ROS 2 drivers, which publish real-time laser scan data on the `/scan` topic. This data is used by SLAM Toolbox for mapping and by Navigation2 for obstacle avoidance and path planning.
 - **More info:**
[SLAMTEC RPLIDAR C1 Product Page](#)
-

WiFi Connection Functionalities in Shell Scripts

Shell scripts can be used to automate WiFi setup and display the robot's IP address on an OLED screen. Here are some typical functionalities:

1. Connecting to WiFi via Shell Script

You can use `nmcli` (NetworkManager CLI) to connect to a WiFi network:

```
#!/bin/bash
# Connect to WiFi
SSID="Your_SSID"
PASSWORD="Your_Password"
nmcli device wifi connect "$SSID" password "$PASSWORD"
```

- This script connects the Raspberry Pi to the specified WiFi network.
- You can run this at boot or manually.

2. Displaying IP Address on OLED via Shell Script

To display the current IP address on an OLED screen, you can use a shell script in combination with Python (for OLED control):

Shell Script Example:

```
#!/bin/bash
# Get the IP address
IP=$(hostname -I | awk '{print $1}')
```

```
# Call Python script to display on OLED  
python3 /home/pi/show_ip_oled.py "$IP"
```

Python Script Example (show_ip_oled.py):

```
import sys  
from some_oled_library import OLED  
  
ip = sys.argv[1]  
oled = OLED()  
oled.display_text(f"IP: {ip}")
```

- The shell script fetches the IP and passes it to the Python script, which handles the OLED display.
 - This can be set to run at startup so the robot always shows its IP on boot.
-